1 Esercizio 1

Abbiamo una funzione f per la quale esiste un valore i tale che $f(i) = 0$. Gli algoritmi che seguono cercano un tale valore i, chiamato *punto zero*, dividendo lo spazio di ricerca in due parti: i punti positivi e i punti negativi.

Un algoritmo è corretto se, per **tutte** le esecuzioni, i due processi finiscono dopo che uno dei due ha trovato un punto zero. Per ciascun algoritmo A, B, C e D determinate se è corretto o meno, giustificando la vostra risposta.

Per gli algoritmi C e D usiamo un'istruzione speciale: `await C then R` che viene eseguita in un modo atomico e ha il seguente comportamento: se la condizione `C` è vera, allora il processo esegue `R` e può proseguire la sua esecuzione, se invece la condizione è falsa, il processo viene interrotto e potrà rifare più tardi l'istruzione.

```
bool found=false;
```

```
Process P
int i=0;
while(!found){
  i=i+1;
  found=(f(i)==0);
}
```

```
Process Q
int j=1;
while(!found){
  j=j-1;
  found=(f(j)==0);
}
```

Figura 1: Algoritmo A

```
bool found=false;
```

```
Process P
int i=0;
while(!found){
  i=i+1;
  if(f(i)==0){
    found=true;
  }
}
```

```
Process Q
int j=1;
while(!found){
  j=j-1;
  if(f(j)==0){
    found=true;
  }
}
```

Figura 2: Algoritmo B

```
bool found=false;
int turn=1;
```

```
Process P
int i=0;
while(!found){
  await turn==1 then turn=2;
  i=i+1;
  if(f(i)==0){
    found=true;
  }
}
```

```
Process Q
int j=1;
while(!found){
  await turn==2 then turn=1;
  j=j-1;
  if(f(j)==0){
    found=true;
  }
}
```

Figura 3: Algoritmo C

esercizio 1

Algoritmo A:

Se per esempio abbiamo $f(1)=0$ e $f(i)\neq 0$ per tutti $x\neq 1$

- Processo P si esegue fino a found = $(f(i)==0)$ con $i=1$
- Processo Q si esegue fino a found = $(f(j)==0)$ con $j=0$

P si esegue e mette found a true, Q si esegue e mette found a false, quindi nessuno termina.
→ Algoritmo A non è corretto

Algoritmo B:

Se prendiamo ancora $f(1)=0$ e $f(x)\neq 0$ per tutti $x\neq 1$

L'algoritmo B non è corretto perchè se lo scheduler esegue per sempre solo Q, non si trova mai il punto zero.

Se prendiamo equità fra i processi, l'algoritmo B è corretto perchè un giorno found passerà a true e il suo valore non cambierà più.
→ Quindi P finirà e Q finirà.

Algoritmo C/D:

riempo $f(1)=0$ e $f(x)\neq 0$ per tutti $x\neq 1$
- Possiamo bloccare Q sull'await
  i P si esegue e si ferma a $i=i/1$ con $i=0$ (e turn vale 2)
- Q si esegue, pausa l'await, mette turn a 1
  mette $j=0$, passa l'if senza cambiare found e torna nell'await con turn = 1
- P si esegue, mette i a 1, mette found a true ed esce

→ Q bloccato nell'await !!!

```
bool found=false;
int turn=1;
```

```
Process P
int i=0;
while(!found){
  await turn==1 then turn=2;
  i=i+1;
  if(f(i)==0){
    found=true;
  }
}
turn=2;
```

```
Process Q
int j=1;
while(!found){
  await turn==2 then turn=1;
  j=j-1;
  if(f(j)==0){
    found=true;
  }
}
turn=1;
```

Figura 4: Algoritmo D

2 Esercizio 2

Consideriamo la soluzione seguente per il problema della sezione critica proposto dai ricercatori Manna e Pnueli.

```
int wantp=0;
int wantq=0;
```

```
Process P
while(true){
  SNC
  if(wantq==-1){
    wantp=-1;
  }else{
    wantp=1;
  }
  while(wantp==wantq){}
  SC
  wantp=0;
}
```

```
Process Q
while(true){
  SNC
  if(wantp==-1){
    wantq=1;
  }else{
    wantq=-1;
  }
  while(wantq==-wantp){}
  SC
  wantq=0;
}
```

Figura 5: Algoritmo di Manna-Pnueli

1. Costruire il diagramma degli stati per questo algoritmo.
2. Dedurre Quale sono le proprietà verificate da questo algoritmo (mutua esclusione, assenza di deadlock, assenza di starvation con fairness fra i processi)?

3 Esercizio 3

Supponiamo che abbiamo a disposizione l'istruzione atomica che segue:

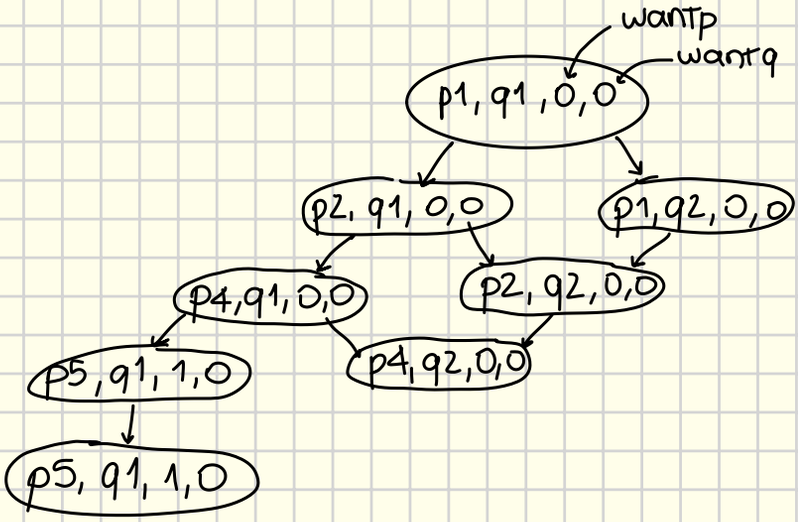
```
int flip(){
  atomic{
    x=(x+1)%2; //inversione della variabile globale x
    return x;  //ritorna il nuovo valore
  }
}
```

Consideriamo la seguente soluzione per risolvere il problema della sezione critica.

1. Dimostrate perché questo algoritmo non rispetta la proprietà di mutua esclusione.

esercizio 2

```
        P                              Q
    While (true) {                 While (true) {
p1      SNC                    q1      SNC
p2      if                     q2      if
p3        wantp=-1             q3        wantq=1
p4      else wantp=1           q4      else wantq=-1
p5    while ( ) {}             q5    while ( ) {}
p6      SC                     q6      SC
p7      wantp=0                q7      wantq=0
```



Non abbiamo la mutua esclusione

P	Q	wantp	wantq
$p_{1\downarrow}$	q1	0	0
$p_{4\downarrow}$	$q_{1\downarrow}$	0	0
$p_{4\downarrow}$	$q_{4\downarrow}$	0	0
$p_{4\downarrow}$	$q_{5\downarrow}$	0	-1
$p_{4\downarrow}$	$q_{6\downarrow}$	0	-1
$p_{5\downarrow}$	q6	1	-1
p6	q6	1	-1

P e Q sono in SC

Mostriamo l'assenza di deadlock:

Ragioniamo per assurdo supponendo che c'è un deadlock → P e Q sono bloccati per sempre nel pre-protocollo! Dove possono essere bloccati per sempre?
P e Q sono bloccati sul loro while → abbiamo wantp==wantq e wantq == -wantp, quindi
wantp == -wantp, quindi wantp=0 e wantq=0
Ma se P è nel while wantp=1 o wantp=-1 } CONTRADDIZIONE.
e se Q è nel while wantq=1 o wantq=-1 } non c'è deadlock

Per dimostrare l'assenza di starvation, bisogna considerare 4 casi:
① P bloccato nel while e wantp=1 per sempre (come P è bloccato non cambia wantp)
② P bloccato nel while e wantp=-1
③ Q bloccato nel while e wantq=1
④ Q bloccato nel while e wantq=-1

① Assumiamo equità fra i processi, altrimenti P esegue il suo while per sempre da solo e rimane bloccato. P bloccato e wantp=1 e ogni volta che fa il test del while, wantq=1.
→ come non c'è deadlock, Q esegue infinitamente spesso un ciclo pre-prot → sc → post-prot, ma allora un giorno farà if con wantp=1 e metterà wantq a 1, bloccherà nel while e sbloccherà P
↓
CONTRADDIZIONE con P sempre bloccato
→ stesso ragionamento con ②, ③ e ④
→ ASSENZA di starvation (se equità tra i processi)

```
int x=0;
```

```
Process P
while(true){
  SNC
  while(flip()!=1){
    while(x!=0){}
  }
  SC
  x=0;
}
```

```
Process Q
while(true){
  SNC
  while(flip()!=1){
    while(x!=0){}
  }
  SC
  x=0;
}
```

Figura 6: Algoritmo di mutua esclusione usando `flip`

2. Cosa cambia se nella funzione `flip`, il `%2` è sostituito da `%3`?
3. Per questa ultima soluzione, giustificare se l'algoritmo rispetta le proprietà di assenza di deadlock e di starvation (con ipotesi di fairness fra i processi).

4 Esercizio 4

Consideriamo la seguente soluzione per risolvere il problema della sezione critica per N processi dove abbiamo un processo S in più che aiuta la gestione.

```
int req=0;
int aut=0;
bool end=true;
```

```
Process P[id] //id in {1,...,N}
while(true){
  SNC
  while(aut!=id){
   req=id;
  }
  SC
  end=true;
  req=0;
}
```

```
Process S
while(true){
  while(req==0){}
  end=false;
  aut=req;
  while(!end){}
  aut=0;
}
```

Figura 7: Algoritmo di mutua esclusione per N processi

1. Dimostrare perché l'algoritmo della figura 7 non rispetta la mutua esclusione.
2. Modifichiamo l'algoritmo per ottenere quello della figura 8. Questo algoritmo verifica la proprietà di mutua esclusione? Qual è il problema di questo ultimo algoritmo?

esercizio 3

```
        P
    While (true){
p1      SNC
p2      while (flip()!=1){
p3          while (x!=0){}
p4      SC
p5      x=0
```

```
        Q
    While (true){
q1      SNC
q2      while (flip()!=1){
q3          while (x!=0){}
q4      SC
q5      x=0
```

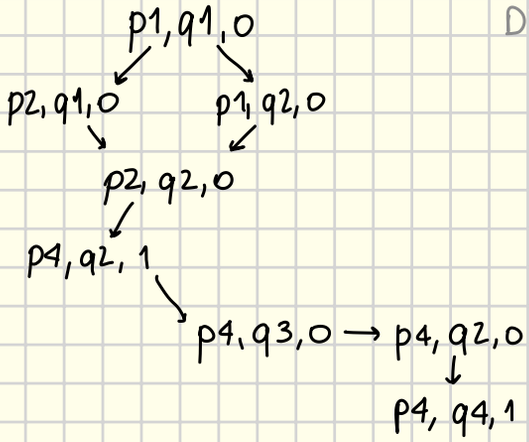


Diagramma degli stati

→ P e Q sono in SC
→ la mutua esclusione non è rispettata

```
int req=0;
int aut=0;
bool end=true;
```

```
Process P[id] //id in {1,...,N}
while(true){
  SNC
  while(aut!=id){
   req=id;
  }
  SC
  end=true;
  req=0;
  while(aut!=0){}
}
```

```
Process S
while(true){
  while(req==0){}
  end=false;
  aut=req;
  while(!end){}
  aut=0;
}
```

Figura 8: Algoritmo di mutua esclusione per N processi